IEEE-754 Single Precision Floating Point Arithmetic Unit

Anuradha D.M.B.P. – 220036L
Department of Electronics and Telecommunication Engineering

December 28, 2025

Contents

1 Introduction **3**

2 System Architecture **3**
2.1 Top-Level Design 3
2.2 Block Design Description 3
2.3 Top-Level Architecture 4

3 Floating-Point Adder/Subtractor **4**
3.1 Functional Description 4
3.2 Special Case Handling 5
3.3 Implementation Nature 5

4 Floating-Point Multiplier **5**
4.1 Functional Description 5
4.2 Special Case Handling 6
4.3 Implementation Nature 6

5 Floating-Point Divider **6**
5.1 Functional Description 6
5.2 Special Case Handling 6
5.3 Implementation Nature 7

6 Asynchronous Design Characteristics **7**
6.1 Advantages 7
6.2 Limitations 7

7 Timing Analysis **7**
7.1 Critical Path 8
7.2 Delay Breakdown 8
7.3 Clock Frequency and VIO Impact 8
7.4 Timing Report Snapshot 8

8 Simulation Results **9**

9 Resource Utilization **9**

10 Conclusion **9**

11 Future Improvements **10**

1 Introduction

Floating-point arithmetic units are essential components in modern digital systems, particularly in signal processing, scientific computation, and embedded control applications. IEEE-754 2008 defines standardized formats and behaviors for floating-point operations, ensuring numerical consistency across platforms.

This project implements an IEEE-754 2008 single-precision floating-point arithmetic unit supporting addition, subtraction, multiplication, and division. The arithmetic logic is implemented asynchronously using combinational logic, while a clocked output stage is employed to provide stable and synchronized outputs.

2 System Architecture

2.1 Top-Level Design

The top-level module `all_together` instantiates three independent arithmetic units:

- Floating-point Adder/Subtractor
- Floating-point Multiplier
- Floating-point Divider

All arithmetic units operate in parallel on the same input operands. A clocked multiplexer selects the desired result based on the operation control signal.

2.2 Block Design Description

Figure 1 shows the Vivado block design used to verify the floating-point unit on hardware. The top-level floating-point module is instantiated within the block design and connected to a Virtual Input/Output (VIO) core. The VIO module provides input operands (`A`, `B`), operation select signals, and control inputs through the JTAG interface, enabling real-time control and observation without external I/O pins.

The output result and IEEE-754 status flags generated by the floating-point unit are routed back to the VIO for monitoring using the Vivado Hardware Manager. This internal debugging approach allows flexible testing of different arithmetic operations and special cases while simplifying hardware integration and validation.

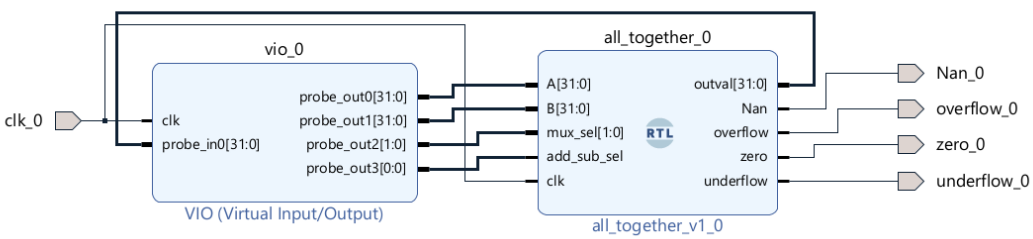


Figure 1: Vivado block design showing the floating-point unit connected to the VIO core for internal debugging

2.3 Top-Level Architecture

Figure 2 shows the internal architecture of the top-level module (`all_together`). The two IEEE-754 input operands are supplied in parallel to the adder/subtractor, multiplier, and divider units. All arithmetic blocks are implemented as combinational logic and operate concurrently.

A multiplexer selects the appropriate arithmetic result based on the operation control signal. The selected output and its corresponding IEEE-754 2008 status flags are registered on the rising edge of the system clock. This architectural separation between combinational computation and synchronous output registration ensures stable operation and predictable timing behavior.

Table 1: Control signal mapping for floating-point operations

Operation	mux_sel	add_sub_sel	Selected Functional Unit
Addition	2'b00	1'b0	Adder/Subtractor (Add Mode)
Subtraction	2'b00	1'b1	Adder/Subtractor (Subtract Mode)
Multiplication	2'b01	X	Multiplier
Division	2'b10	X	Divider

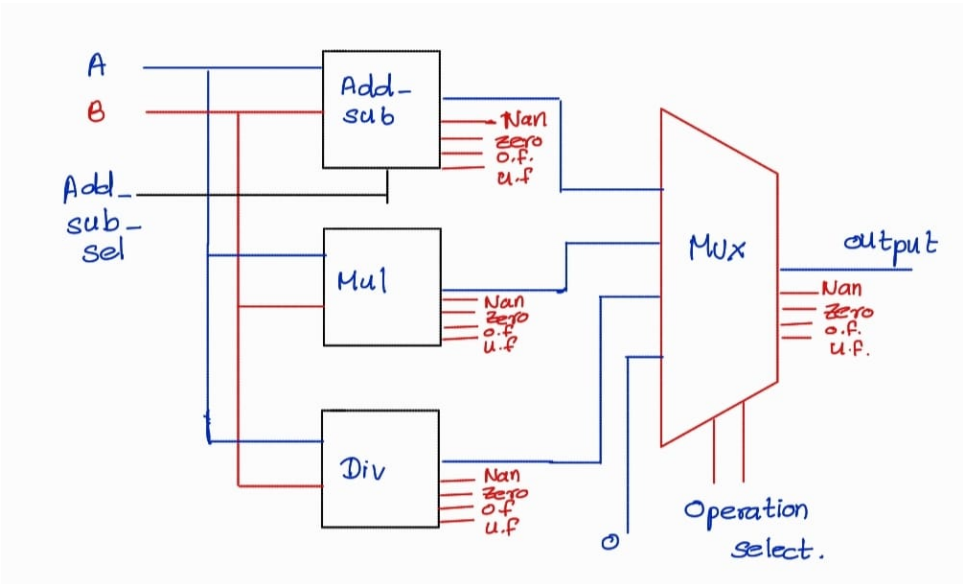


Figure 2: Internal architecture of the `all_together` floating-point unit

This design choice ensures glitch-free output while maintaining simplicity.

3 Floating-Point Adder/Subtractor

3.1 Functional Description

The floating-point adder/subtractor module performs IEEE-754 2008single-precision addition and subtraction based on the `add_sub_sel` control signal. The operation is carried out through the following steps:

1. Input operands are decoded into sign, exponent, and mantissa fields, and special cases are identified.
2. Exponents are compared, and the smaller mantissa is right-shifted to achieve alignment.
3. Mantissas are added or subtracted using two's complement arithmetic depending on the effective operation.
4. The result is normalized, and the exponent is adjusted accordingly before recombination.

3.2 Special Case Handling

The module includes dedicated logic to correctly handle IEEE-754 2008 special conditions:

- NaN inputs propagate a NaN result.
- Infinity operands are processed according to IEEE-754 arithmetic rules.
- Zero results are detected after normalization.
- Overflow and underflow conditions are flagged based on exponent range checks.

3.3 Implementation Nature

The adder/subtractor is implemented as a fully combinational design using an `always @(*)` block. Normalization and alignment operations are implemented using bounded `while` loop, which are synthesized as parallel priority logic rather than iterative hardware, contributing to the overall combinational delay.

4 Floating-Point Multiplier

4.1 Functional Description

The floating-point multiplier module performs IEEE-754 2008 single-precision multiplication on two input operands. The operation is executed through the following stages:

1. The input operands are decomposed into sign, exponent, and mantissa components, and special values are detected.
2. The result sign is computed using the XOR of the operand signs, while the exponents are added with bias correction.
3. The mantissas, including the implicit leading one for normalized numbers, are multiplied to produce an extended precision product.
4. The product is normalized, and the exponent is adjusted to generate the final IEEE-754 formatted result.

4.2 Special Case Handling

The multiplier includes logic to correctly handle IEEE-754 2008 exceptional conditions:

- NaN inputs produce a NaN result.
- Multiplication involving infinity and zero follows IEEE-754 rules.
- Zero results are detected from mantissa evaluation.
- Overflow and underflow are flagged based on exponent limits after normalization.

4.3 Implementation Nature

The multiplier is implemented as a fully combinational circuit using `always @(*)` blocks. Mantissa multiplication and normalization are performed using fixed-width arithmetic.

5 Floating-Point Divider

5.1 Functional Description

The floating-point divider module performs IEEE-754 2008 single-precision division of two input operands. The operation proceeds through the following steps:

1. The operands are decomposed into sign, exponent, and mantissa fields, and special cases such as zero, infinity, and NaN are detected.
2. The result sign is computed as the XOR of the operand signs, and the exponent is calculated by subtracting the divisor exponent from the dividend exponent with bias adjustment.
3. The mantissas are divided using extended precision arithmetic to produce a quotient with sufficient bits for normalization.
4. Leading zeros in the quotient are counted to normalize the result, and the exponent is adjusted accordingly before forming the final IEEE-754 output.

5.2 Special Case Handling

The divider module includes logic to handle IEEE-754 2008 special cases correctly:

- Division by zero generates infinity with the appropriate sign.
- Zero numerator produces zero as a result.
- Infinity divided by a finite number, and operations involving NaN, are handled according to IEEE-754 rules.
- Overflow and underflow conditions are flagged based on the normalized exponent range.

5.3 Implementation Nature

The divider is implemented as a fully combinational design using an `always @(*)` block. Mantissa division, leading-zero detection, and normalization are performed using fixed-width arithmetic and bounded loops, which are synthesized as parallel combinational logic, forming the critical path for propagation delay.

6 Asynchronous Design Characteristics

6.1 Advantages

- Arithmetic results are generated immediately after input changes, without waiting for a clock edge.
- Parallel operation of adder, multiplier, and divider simplifies overall control logic.
- Clear separation between combinational computation and synchronous output registration improves design clarity.
- Suitable for functional verification and rapid prototyping using internal debug tools such as VIO.

6.2 Limitations

- Long combinational paths, especially in the divider, increase overall propagation delay.
- The maximum operating frequency is limited by the worst-case combinational delay.
- Power consumption may increase due to continuous switching in combinational logic.
- Scalability is limited for high-performance or deeply pipelined designs.

7 Timing Analysis

Timing analysis was performed using Xilinx Vivado after synthesis and implementation. The design follows a hybrid architecture in which all floating-point arithmetic units operate combinationally, while the final result and status flags are registered at the top-level output. As a result, the maximum operating frequency is determined by the longest combinational path from the input source to the output register.

In this implementation, the inputs are driven through a Vivado Virtual Input/Output (VIO) core via the JTAG interface. Therefore, the critical timing path extends from the internal VIO registers, through the selected arithmetic unit, and finally to the output register in the top-level module.

7.1 Critical Path

The critical path identified by Vivado is shown below:

```
VIO Register → Divider Mantissa Division → Normalization Logic → Output Register
```

Among the three arithmetic units, the floating-point divider exhibits the longest combinational delay due to mantissa division, leading-zero detection, exponent adjustment, and normalization logic. Consequently, the divider dominates the overall timing behavior of the floating-point unit.

7.2 Delay Breakdown

The post-implementation timing report indicates a total path delay of:

- **Total Delay:** 185.309 ns
- **Logic Delay:** 76.572 ns
- **Net (Routing) Delay:** 108.737 ns

The logic delay arises primarily from large combinational arithmetic blocks, including division and normalization loops. The remaining delay is attributed to routing between logic elements and registers across the FPGA fabric.

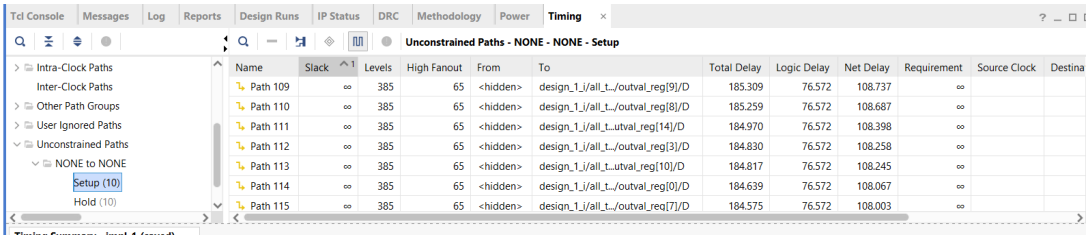
7.3 Clock Frequency and VIO Impact

The design is operated with a system clock frequency of **30 MHz**, corresponding to a clock period of 33.33 ns. Since the arithmetic computation is purely combinational and the output is registered, the functional correctness of the design is maintained despite the long combinational delay by allowing sufficient time between input updates.

The use of the Vivado VIO core through the inbuilt JTAG interface further relaxes real-time timing constraints, as inputs are applied manually and not at full clock rate. This makes the design suitable for functional verification and debugging, even though the measured critical path delay exceeds a single clock period at higher frequencies.

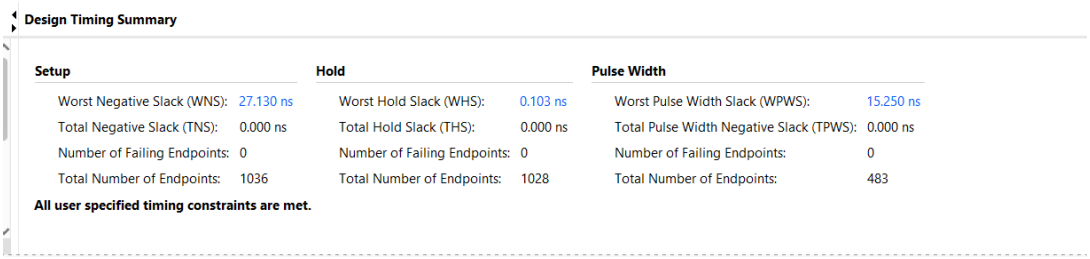
7.4 Timing Report Snapshot

Figure 3 shows the Vivado timing analysis highlighting the critical path from the VIO input registers to the output register.



Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destina
Path 109	∞	385	65	<hidden>	design_1_i/all_t.../outval_reg[9]/D	185.309	76.572	108.737	∞		
Path 110	∞	385	65	<hidden>	design_1_i/all_t.../outval_reg[8]/D	185.259	76.572	108.687	∞		
Path 111	∞	385	65	<hidden>	design_1_i/all_t...utval_reg[14]/D	184.970	76.572	108.398	∞		
Path 112	∞	385	65	<hidden>	design_1_i/all_t.../outval_reg[3]/D	184.830	76.572	108.258	∞		
Path 113	∞	385	65	<hidden>	design_1_i/all_t...utval_reg[10]/D	184.817	76.572	108.245	∞		
Path 114	∞	385	65	<hidden>	design_1_i/all_t.../outval_reg[0]/D	184.639	76.572	108.067	∞		
Path 115	∞	385	65	<hidden>	design_1_i/all_t.../outval_reg[7]/D	184.575	76.572	108.003	∞		

Figure 3: Post-implementation timing report showing the critical path from VIO to output register



Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	27.130 ns	Worst Hold Slack (WHS):	0.103 ns	Worst Pulse Width Slack (WPWS):	15.250 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	1036	Total Number of Endpoints:	1028	Total Number of Endpoints:	483

All user specified timing constraints are met.

Figure 4: Post-implementation timing report showing the summary for VIO

8 Simulation Results

Figure 5 shows the simulation waveform results demonstrating correct behavior of the floating-point unit under various test cases.

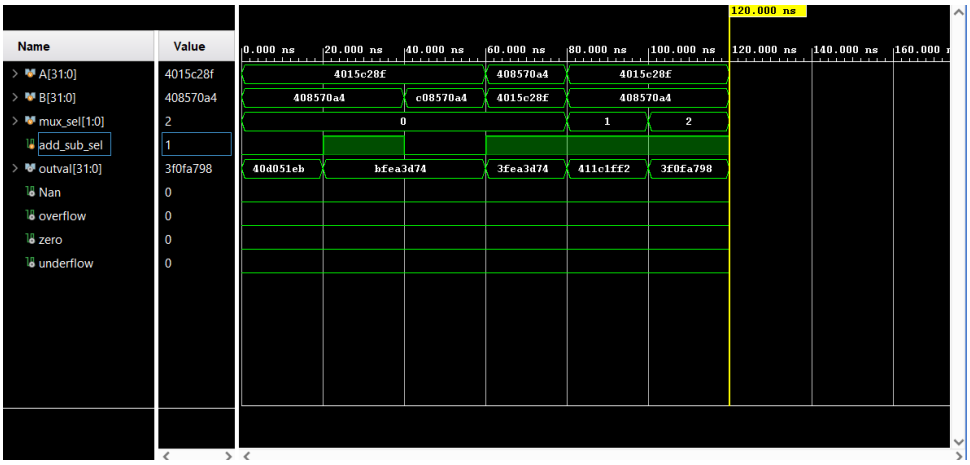


Figure 5: Simulation results verifying IEEE-754 2008 compliance for normal, subnormal, zero, infinity, and NaN cases

9 Resource Utilization

Resource utilization was obtained from the Xilinx Vivado post-synthesis report. Figure 6 summarizes the FPGA resources consumed by the floating-point arithmetic unit, including lookup tables (LUTs), flip-flops (FFs), and other relevant logic resources.

10 Conclusion

This project successfully demonstrates an asynchronous IEEE-754 2008 single-precision floating-point arithmetic unit implemented in Verilog HDL. While the asynchronous arithmetic cores provide simplicity and clarity, the divider introduces a significant combinational delay that limits maximum clock frequency. The design serves as a strong educational reference for floating-point hardware design and FPGA synthesis.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	3360	17600	19.09
LUTRAM	24	6000	0.40
FF	1378	35200	3.91
DSP	2	80	2.50
IO	5	100	5.00
BUFG	2	32	6.25

Figure 6: Vivado resource utilization report for the IEEE-754 2008 floating-point unit

11 Future Improvements

- Introduce pipelining for divider and multiplier
- Support IEEE-754 2008 rounding modes
- Convert to a fully synchronous multi-cycle FPU
- Add valid/ready handshaking